Algoritmalar ve Veri Yapıları Bootcamp Ödev 2.
Çözümler

1. **a.**

```
sum ← 0
for j ← n downto 1 step j/2 do
        for i ← 0  to j-1 do
                sum ← sum + 1

                        d ← d - 1
```

Yukarıdaki algoritmada iki iç içe döngü olması ilk etapta algoritmanın zaman karmaşıklığının $O(n^2)$ olduğunu düşündürse de, detaylı inceleme farklı bir sonuca işaret etmektedir. Şöyle ki: Dış döngü değişkeni n'den başlayarak 1'e doğru değerler almakta ve döngünün her bir iterasyonunda döngü değişkeninin aldığı değer bir önceki iterasyonda aldığı değerin yarısına eşit olmaktadır. Dış döngünün her bir turunda ve de dış döngü değişkeninin aldığı her bir değerde iç döngü ise o değer kadar dönmektedir. Dolayısıyla, algoritmanın toplam iterasyon ve dolayısıyla da en iç bölgedeki toplama işleminin koşturulma adedi şu şekilde bir seri izlemektedir:

n + n/2 + n/4 + n/8 + …….. + 1

Bu da;

n * (1 + ½ + ¼ + ⅛ + ……..+ 1/n)

eşittir. Parantez içindeki serinin, her bir terim eklendikten sonra aldığı toplam değerleri düşünürsek ardışık değerlerini düşünürsek:

1 -> 1.5 → 1.75 → 1.875 → ……. →1.99999999

DOlayısıyla, parantez içerisindeki seri n sonsuza doğru gittiğinde 2'ye yakısayacaktır. Bu da yukarıda verilen algoritma parçasının zaman karmaşıklığını

T(n) is *O(2n)* yani
T(n) is ***O(n) (lineer)***

yapacaktır.

b.

```
sum ← 0
for i ← 1 to n-1 step i*2 do
    for j ← 0 to i-1 do
        sum← sum + 1
```

Yine iç içe iki döngü ve ilk bakışta $O(n^2)$ gibi görünen bir algoritma ile karşı karşıyayız. Detaylı inceleyelim. Dış döngü değişkeni 1'den başlayarak n-1'e doğru değerler almakta ve döngünün her bir iterasyonunda döngü değişkeninin aldığı değer bir önceki iterasyonda aldığı değerin iki katına eşit olmaktadır. Dış döngünün her bir turunda ve de dış döngü değişkeninin aldığı her bir değerde iç döngü ise o değer kadar dönmektedir. Dolayısıyla, algoritmanın toplam iterasyon ve dolayısıyla da en iç bölgedeki toplama işleminin koşturulma adedi şu şekilde bir seri izlemektedir:

$1 + 2 + 4 + 8 + \ldots\ldots\ldots\ldots\ldots + n$

Bu da, 1'den bir üst sınıra kadar olan 2'nin ardışık üstlerinin toplam değerine eşittir. Yani (her bir turda iki katına çıkan bir değerin, logn kadar tur sonunda n değerine ulaşacağı düşünülürse);

$2^0 + 2^1 + 2^2 + 2^3 + \ldots\ldots\ldots\ldots\ldots + 2^{\log n}$

Bu seri de, tanım gereği

$2^{\log n+1} - 1$

Değerin eşittir. Dolayısıyla, algoritmamızın zaman karmaşıklığı:

$$T(n) = 2^{\log n+1} - 1$$
$$= 2.2^{\log n} - 1$$
$$= 2n - 1$$

Düşük seviyeli terimleri ve katsayıları düşürdükten sonra;

T(n) is ***O(n)*** yani ***lineer***

b.

```
sum ← 0
for  i ← 1 to n-1 step i*2  do
      for j ← 0 to n-1  do
           sum← sum + 1
```

Yine iç içe iki döngü içeren bir algoritma parçasıyla karşı karşıyayız. İnceleyelim: Dış döngü değişkeni 1'den başlayarak n-1'e doğru değerler almakta ve döngünün her bir iterasyonunda döngü değişkeninin aldığı değer üstteki (b) şıkkında olduğu gibi bir önceki iterasyonda aldığı değerin iki katına eşit olmaktadır. Dış döngünün her bir turundaysa iç döngü n kadar dönmektedir ve dönüş adedi dış döngü değişkeninin aldığı değerden bağımsızdır. Dolayısıyla, algoritmanın toplam iterasyon ve dolayısıyla da en iç bölgedeki toplama işleminin koşturulma adedi şu şekilde bir seri izlemektedir:

n + n + n + …….. + n

Yukarıdaki serinin her biri n olan terimlerinin toplam adedi, dış döngünün toplam iterasyon adedi olan logn'e eşittir (her bir turda iki katına çıkan dış döngü değişkenin değerinin logn tur sonra n değerine ulaşacağını göz önüne alarak). Dolayısıyla;

T(n) = n.logn

T(n) is ***O(nlogn)***

2. a. Her iki fonksiyon da 2. dereceden polinom olduğundan, polinomlar için geçerli olan genel kuralları büyüme derecesi sınıflandırmalarına uygulayarak, her iki fonksiyonun da aynı büyüme derecesi sınıfından olduğunu, yani f(n)'nin O(g(n))'de, g(n)'nin de O(f(n)) olduğunu iddia edebiliriz. Yani, bir başka ifadeyle:

f(n) is Θ(g(n))

İddiamızı kanıtlamak için öncelikle iki fonksiyonun eşit değere sahip olduğu n değerini bulalım.

f(n) = g(n)
n(n+1) = 2000 n^2

n = 1/1999

Yukarıdaki değere en yakın ve daha büyük tam sayı değerini, yani 1'i, eşik değer (n_0) olarak alalım. n değerlerinin 1'den büyük olduğu durumlarda fonksiyonların değerleri şöyle olacaktır:

n	*f*	*g*
1	*1*	*2000*
2	*6*	*8000*
3	*12*	*18000*
4	*20*	*32000*

c = 1 için, aşağıdaki eşitsizlik:

f(n)≤ c g(n) her bir n ≥ 1 değeri için

c = 2000 için de aşağıdaki eşitisizlik, hep geçerli olacaktır:

f(n)≥ c g(n) her bir n ≥ 1 değeri için

Dolayısyla**,**

f(n) is O(g(n))
g(n) is O(f(n))

Yani***;***

f(n) is Θ(g(n)) (ya da alternatif olarak: g(n) is Θ(f(n))

b. f(n) ikinci dereceden bir polinom iken g(n) üçüncü dereceden bir polinom olduğundan, polinomların büyüme derecesi sınıflandırmalarına ilişkin genel kuralları uygulayarak, f'nin en fazla g kadar hızlı büyüdüğünü, ancak daha hızlı büyümediğini, aslında n sonsuza yaklaşırken asimptotik olarak g'den daha yavaş büyüdüğünü iddia edebiliriz. Yani, f(n) O(g(n))'dedir. İddiamızı kanıtlamak için öncelikle iki fonksiyonun eşit değere sahip olduğu n değerini bulalım.

f(n) = g(n)
$100n^2 = 0{,}01\ n^3$

n = 10000

n değerleri 10000'den büyük olduğunda fonksiyonların değerleri şöyledir:

n	*f*	*g*
10000	*10000000000*	*10000000000*
10001	*10002000100*	*10003000300*
10003	*10006000900*	*10009002700*
10004	*10008001600*	*10012004800*

Yukarıdaki tablodan da görülebileceği gibi, c = 1 için her n ≥ 10000 için aşağıdaki eşitsizlik geçerlidir.

f(n) ≤ c g(n) her bir n ≥ 10000 değeri için

Dolayısıyla*;*

f(n) is O(g(n))

c. Bu durumu limitler kullanarak ele almak daha iyi olacaktır:

$$\lim_{n\to\infty} f(n)/g(n) = \lim_{n\to\infty} (n-1)!/n!$$

$$= \lim_{n\to\infty} (n-1)!/n*(n-1)!$$

$$= \lim_{n\to\infty} 1/n$$

$$= 0$$

Dolayısıyla, limitler kullanarak büyüme düzenini belirleme tanımı gereği:

f(n) is O(g(n))

d. Bunu da limitler kullanarak ele alırsak:

$$\lim_{n\to\infty} f(n)/g(n) = \lim_{n\to\infty} \log^2 n/\log n^2$$

L'Hopital kuralını uygulayabilir ve limitteki her iki fonksiyonu da türevleriyle değiştirebiliriz.

$$= \lim_{n\to\infty} (\log^2 n)' / (\log n^2)'$$

(log(n))²'nin n'ye göre türevi:

$$(\log^2 n)' = (2\log n)/n$$

Öte yandan, logaritmanın tanımı gereği,

$$\log n^2 = 2 \log n$$

Dolayısıyla, log n²'nin n'ye göre türevi, 2logn'nin n'ye göre türevine eşittir. logn'nin n'ye göre türevi ise şöyledir:

$$\log(n)' = 1/n$$

Dolayısıyla, 2logn'nin n'ye göre türevi:

$$(2\log(n))' = 2/n$$

Yani, log n²'nin n'ye göre türevi şöyledir:

$$(\log n^2)' = 2/n$$

Orijinal fonksiyonları, limit durumunda türevleriyle değiştirdiğimizde:

$$\lim_{n\to\infty} f(n)/g(n) = \lim_{n\to\infty} f'(n)/g'(n)$$

$$= \lim_{n\to\infty} ((2\log n)/n)/(2/n)$$

$$= \lim_{n\to\infty} \log n$$

n sonsuza yaklaştıkça, doğal logaritma logn de sonsuza yaklaşır. Ancak, n'nin kendisinden daha yavaş bir şekilde sonsuza yaklaşır.

Dolayısıyla;

$$\lim_{n\to\infty} f(n)/g(n) = \infty$$

Bu bize, limitler kullanarak büyüme düzenini belirlemenin tanımı gereği şunu söylüyor:

g(n) is O(f(n))

e. İlk gözlem bize f(n) = 2*g(n) olduğunu söylüyor. Her ikisi de birbirinin sabit bir katı olduğundan, aynı büyüme sınıfında olmaları gerekir. Şimdi n'nin 1'den büyük değerleri için her iki fonksiyonun değerlerine bakalım.

n	*f*	*g*
1	*2*	*1*
2	*4*	*2*
3	*8*	*4*
4	*16*	*8*

for c = 1/3 aşağıdaki eşitsizlik:

$$f(n) \leq c\ g(n) \quad \text{her bir } n \geq 1 \quad \text{için}$$

ve c = 1 için de aşağıdaki eşitsizlik :

$$f(n) \geq c\ g(n) \quad \text{her bir } n \geq 1 \quad \text{için}$$

n'nin 1'den büyük tüm değerleri için geçerli olacaktır. Dolayısıyla:

f(n) is O(g(n))
g(n) is O(f(n))

Yani*;*

f(n) is Θ(g(n)) (ya da alternatif olarak: g(n) is Θ(f(n))

f. Logaritmanın tanımı gereği:

$\text{Log}_2 n = \ln n / \ln 2$

Dolayısıyla:

$f(n) = 1/\ln 2 * g(n)$

Her iki fonskiyonun birbirleriyle bir sabit katsayıya dayanan eşitlik ilişkisi olması zaten her iki fonksiyonunda aynı "order-of-growth" sınıfına dahil olduğunu gösteriyor. Ama bunu kanıtlamak için

$c = 1/\ln 2$ ve $n_0 = 1$ değerlerini alırsak;

$c.g(n) \leq f(n) \leq c.g(n)$ her bir $n \geq 1$ için

Eşitsizliklerinin her ikisinin de geçerli olduğunu görebiliriz. Bu da her iki fonksiyonun da birbirlerinin ***big-Oh***'su olduğu anlamına gelir (*aynı c değerini yukarıda her iki eşitsizlikte de kullanmanın önünde bir engel bulunmamakta zira eşitsizlikler eşitlik durumlarını da içermekte.*)

Aynı sonuca limitler kullanarak erişmeyi denersek:

$$\lim_{n\to\infty} f(n)/g(n) = \lim_{n\to\infty} \log_2 n/\ln n$$

$$= \lim (\ln n/\ln 2)/\ln n$$

$$= \lim 1/\ln 2$$

$$= 1/\ln 2$$

$$= 1.4427$$

Elde edilen limit pozitif bir tam sayı olduğundan, her iki fonksiyonun da birbirlerine denk bir "order-of-growth" karakteri gösterdiğini söyleyebiliriz. Sonuç olarak:

f(n) is O(g(n))
g(n) is O(f(n))

Yani***;***

f(n) is Θ(g(n)) ***(ya da alternatif olarak: g(n) is Θ(f(n))***

Yukarıdaki sorulara verdiğimiz cevaplarla aşağıdaki tabloyu doldurursak:

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$f(n) = n(n+1)$ $g(n) = 2000n^2$	✅	✅
$f(n) = 100n^2$ $g(n) = 0{,}01\ n^3$	✅	❌
$f(n) = (n-1)!$ $g(n) = n!$	✅	❌
$f(n) = \log^2 n$ $g(n) = \log n^2$	❌	✅
$f(n) = 2^n$ $g(n) = 2^{n-1}$	✅	✅
$f(n) = \log n$ *(Logarithm base 2)* $g(n) = \ln n$ *(Logarithm base e)*	✅	✅